Documentación de Acciones Correctivas, Preventivas y de Mejoramiento (Mejora Continua)

Proyecto EcoRuteando — Plataforma de Movilidad Sostenible

Control de versiones del documento

| Versión | Fecha | Autor | Descripción del cambio | |---------|-------|-------|----------------------| | 1.0 | 13 de octubre de 2026 | Equipo de Arquitectura y QA | Emisión inicial del plan de mejora continua |

Metadatos del proyecto

| Campo | Valor | |-------|-------| | **Nombre del proyecto** | EcoRuteando | | **Descripción** | Plataforma de movilidad sostenible para planificación de rutas ecológicas, estimación de impacto ambiental y reporte de obstáculos urbanos | | **Backend** | .NET 9 (C#) — 23 proyectos, arquitectura DDD + Clean Architecture con CQRS/MediatR | | **Frontend Web** | React 18 + Vite 8 + Tailwind CSS (SPA) | | **Aplicación Móvil** | Expo SDK 57 / React Native 0.86 | | **Base de Datos** | PostgreSQL 16 + PostGIS 3.5 + Liquibase 5.0.2 | | **Métodos de autenticación soportados** | Email/contraseña + OAuth Google | | **Bounded Contexts** | Security, Mobility, Notifications, Admin, Integrations | | **Repositorios** | EcoRuteandoBackend, EcoRuteando (frontend), Movil-EcoRuteando, EcoRuteandoDb | | **Estado actual del proyecto** | En desarrollo activo, ~87% de funcionalidades completadas | | **Clasificación del documento** | Documentación oficial de calidad — Confidencial interno |

---

1. Introducción y objetivos

1.1 Introducción

El presente documento consolida el plan de **mejora continua** del proyecto EcoRuteando, resultado de una auditoría técnica integral realizada sobre los cuatro repositorios que componen la plataforma. La auditoría evaluó la arquitectura de software, la calidad del código, la seguridad, la infraestructura de despliegue y los procesos de aseguramiento de calidad.

EcoRuteando es una plataforma de movilidad sostenible que permite a los usuarios planificar rutas ecológicas (a pie, en bicicleta, en transporte público o vehículo particular), estimar su impacto ambiental en términos de emisiones de CO_2 y calorías quemadas, reportar obstáculos en el mapa urbano y recibir alertas climáticas basadas en su ubicación. La plataforma se compone de una API REST en .NET 9, una aplicación web SPA en React, una aplicación móvil en React Native/Expo y una base de datos PostgreSQL con extensión geoespacial PostGIS, gestionada mediante migraciones Liquibase.

Si bien el proyecto ha alcanzado un estado funcional avanzado —con el 87% de las funcionalidades planificadas completamente implementadas y operativas—, la auditoría identificó **brechas críticas en seguridad, calidad de software e infraestructura DevOps** que requieren atención inmediata.

Este documento establece las acciones correctivas, preventivas y de mejoramiento necesarias para elevar el estándar de calidad del proyecto a niveles de producción profesional.

1.2 Objetivos del plan de mejora continua

| # | Objetivo | Prioridad | |---|---------|-----------| | O1 | **Eliminar brechas de seguridad críticas** — Rotar credenciales filtradas, endurecer la gestión de secretos y prevenir futuras exposiciones | Crítica | | O2 | **Restablecer funcionalidades degradadas** — Garantizar que el servicio de correo electrónico (verificación y recuperación de contraseñas) opere correctamente en todos los entornos | Alta | | O3 | **Establecer una cultura de pruebas automatizadas** — Implementar una suite de pruebas que proteja la lógica de negocio y facilite el desarrollo continuo con confianza | Alta | | O4 | **Automatizar la integración y entrega continua** — Configurar pipelines CI/CD que validen, construyan y desplieguen automáticamente cada cambio | Alta | | O5 | **Mitigar riesgos operativos** — Establecer mecanismos de prevención de pérdida de datos, caídas de servicio y regresiones | Media | | O6 | **Optimizar la observabilidad y resiliencia** — Centralizar logs, limitar abusos y habilitar notificaciones push reales | Media | | O7 | **Asegurar la sostenibilidad técnica** — Crear indicadores medibles que permitan evaluar el progreso de las acciones implementadas | Media |

1.3 Alcance del documento

El plan abarca los cuatro repositorios del proyecto:

- **EcoRuteandoBackend** — API .NET 9 con arquitectura DDD + Clean Architecture
- **EcoRuteando** — Frontend React/Vite y orquestación Docker
- **Movil-EcoRuteando** — Aplicación móvil Expo/React Native
- **EcoRuteandoDb** — Esquema de base de datos y migraciones Liquibase

Las acciones están clasificadas en tres categorías:

- **Correctivas** (Sección 2): Resuelven fallas y brechas identificadas actualmente
- **Preventivas** (Sección 3): Mitigan riesgos futuros
- **De mejoramiento** (Sección 4): Optimizan arquitectura y calidad a mediano plazo

---

2. Matriz de Acciones Correctivas

Las acciones correctivas abordan fallas y brechas **críticas actualmente identificadas** que impactan directamente la seguridad, funcionalidad y calidad del producto.

2.1 Acción Correctiva AC-01: Rotación de credenciales y API keys filtradas en el historial de Git

| Campo | Descripción | |-------|-------------| | **ID** | AC-01 | | **Título** | Eliminación de credenciales reales y API keys commiteadas al repositorio | | **Severidad** | 🔴 **Crítica** | | **Estado actual** | 🔴 Abierta — Pendiente de ejecución |

Descripción del problema: Durante la auditoría se detectó que archivos `.env` conteniendo credenciales reales —incluyendo API keys de Google Maps, Client IDs y Client Secrets de OAuth Google, y credenciales SMTP— fueron commiteados al historial de Git de los repositorios. Si bien

los archivos `.env` actuales están correctamente incluidos en `.gitignore`, las versiones históricas permanecen accesibles en el historial de commits para cualquier persona con acceso al repositorio.

Impacto:

- Explotación no autorizada de la API key de Google Maps → generación de facturas inesperadas en Google Cloud Console
- Acceso no autorizado a los servicios OAuth de Google → suplantación de identidad
- Compromiso de credenciales SMTP → envío de correos phishing en nombre de EcoRuteando
- Incumplimiento de principios de seguridad de la OWASP Top 10 (A07: Identification and Authentication Failures)

Causa raíz:

- Falta de un proceso formal de revisión de seguridad antes de commits
- Ausencia de herramientas de detección de secretos (secret scanning) en el flujo de trabajo
- Desconocimiento de la política de gestión de secretos durante las fases iniciales del desarrollo

Solución propuesta:

| Fase | Acción | Herramienta | Responsable | |------|--------|------------|-------------| | Fase 1 (inmediata) | **Rotar todas las credenciales comprometidas** — Generar nuevas API keys en Google Cloud Console, nuevos Client Secrets OAuth y nueva contraseña de aplicación SMTP | Google Cloud Console, Gmail | DevOps / Backend Lead | | Fase 2 (24-48h) | **Limpiar el historial de Git** — Ejecutar `git filter-repo` o BFG Repo-Cleaner para eliminar archivos `.env` del historial completo | `git filter-repo`, BFG | DevOps | | Fase 3 (inmediata) | **Configurar Git Guard/secret scanning** — Habilitar detección automática de secretos en GitHub (Secret Scanning + Push Protection) | GitHub Advanced Security | DevOps | | Fase 4 (1 semana) | **Crear `.env.example` sanitizado** — Verificar que la plantilla no contenga valores reales, solo placeholders | Revisión manual | Backend Lead | | Fase 5 (2 semanas) | **Implementar pre-commit hooks** — Configurar `pre-commit` con `detect-secrets` o `gitleaks` para bloquear commits con secretos | `pre-commit`, `gitleaks` | Equipo completo |

Criterios de aceptación:

- [ ] Ninguna API key, Client Secret o contraseña real presente en el historial de Git (verificado con `gitleaks detect`)
- [ ] Todas las credenciales rotadas y funcionales
- [ ] Secret Scanning habilitado en los 4 repositorios de GitHub
- [ ] Pre-commit hooks configurados y activos en todos los clones locales

Responsable: DevOps Engineer + Backend Lead **Plazo estimado:** 1-2 semanas **Prioridad:** Inmediata — ejecutar antes de cualquier otro trabajo

---

2.2 Acción Correctiva AC-02: Restablecimiento del servicio SMTP en el entorno Docker

| Campo | Descripción | |-------|-------------| | **ID** | AC-02 | | **Título** | Corrección del servicio de envío de correos electrónicos en Docker | | **Severidad** | 🟠 **Alta** | | **Estado actual** | 🔴 Abierta — Pendiente de ejecución |

Descripción del problema: El servicio de correo electrónico (implementado con MailKit y SMTP de Gmail) falla al ejecutarse dentro del stack de Docker. Esto degrada las siguientes funcionalidades:

- **Verificación de email** — Los usuarios nuevos no reciben el correo de verificación
- **Recuperación de contraseña** — Los usuarios no pueden restablecer su contraseña olvidada

Como workaround manual, los usuarios administradores deben marcar manualmente las cuentas como verificadas mediante SQL:

```
UPDATE security.users SET email_verified = true WHERE email = '<email>';
```

Impacto:

- Los usuarios no pueden verificar su cuenta de forma autónoma
- La recuperación de contraseña está inoperativa → usuarios bloqueados sin administrador
- Experiencia de usuario degradada en el flujo de registro
- Dependencia de intervención manual para tareas rutinarias

Causa raíz:

- Las variables de entorno SMTP (`EmailSettings__Host` , `EmailSettings__Port` , `EmailSettings__Username` , `EmailSettings__Password` , `EmailSettings__EnableSsl`) no están correctamente inyectadas en el contenedor del backend
- El `docker-compose.yml` define parcialmente las variables SMTP pero omite `Username` y `Password`
- Posible incompatibilidad de autenticación SMTP dentro de la red aislada de Docker (firewall, TLS handshake)

Solución propuesta:

| Fase | Acción | Responsable | |------|--------|-------------| | Fase 1 | **Auditar la configuración SMTP actual** — Verificar variables de entorno en `.env` y `docker-compose.yml` | Backend Lead | | Fase 2 | **Completar variables SMTP en docker-compose** — Asegurar que `EmailSettings__Username` y `EmailSettings__Password` se pasen desde `.env` via `env_file` | DevOps | | Fase 3 | **Probar conectividad SMTP desde dentro del contenedor** — `docker compose exec backend sh` y usar `telnet smtp.gmail.com 587` o similar | Backend Lead | | Fase 4 | **Implementar reintentos y cola de correos** — Agregar retry policy con Polly y logging de fallos SMTP | Backend Lead | | Fase 5 | **Agregar health check de SMTP** — Endpoint de diagnóstico que verifique la conectividad SMTP sin enviar correos reales | Backend Lead | | Fase 6 | **Configurar fallback** — En caso de fallo SMTP, loggear el error y notificar al administrador via dashboard | Backend Lead |

Criterios de aceptación:

- [ ] El registro de un nuevo usuario envía un correo de verificación exitosamente en Docker
- [ ] La recuperación de contraseña envía el correo de restablecimiento exitosamente
- [ ] Los fallos SMTP se registran en logs con información suficiente para diagnóstico
- [ ] Un health check de SMTP está disponible y reporta estado correcto

Responsable: Backend Lead + DevOps **Plazo estimado:** 1 semana

---

2.3 Acción Correctiva AC-03: Implementación de una suite de pruebas automatizadas

| Campo | Descripción | |-------|-------------| | **ID** | AC-03 | | **Título** | Creación de pruebas automatizadas para backend, frontend y base de datos | | **Severidad** | 🟠 **Alta** | | **Estado actual** | 🔴 Abierta — Pendiente de ejecución |

Descripción del problema: El proyecto **no cuenta con ninguna suite de pruebas automatizadas** en ninguno de sus cuatro repositorios:

| Repositorio | Tipo de prueba | Estado | |------------|---------------|--------| | EcoRuteandoBackend | Unit tests (xUnit/NUnit) | ❌ No existen | | EcoRuteandoBackend | Integration tests | ❌ No existen | | EcoRuteandoBackend | API tests | ❌ No existen | | EcoRuteando (frontend) | Component tests (Jest/RTL) | ❌ No existen | | EcoRuteando (frontend) | E2E tests (Cypress/Playwright) | ❌ No existen | | Movil-EcoRuteando | Unit tests (Jest) | ❌ No existen | | Movil-EcoRuteando | E2E tests (Detox) | ❌ No existen | | EcoRuteandoDb | Migration tests | ❌ No existen |

Impacto:

- Cualquier cambio en el código puede introducir regresiones silenciosas
- Los desarrolladores dependen exclusivamente de pruebas manuales → lento y propenso a errores
- No hay red de seguridad para refactors o migraciones de base de datos
- Dificulta la incorporación de nuevos desarrolladores al proyecto
- Imposibilidad de habilitar CI/CD efectivo (no hay nada que ejecutar en el pipeline)

Causa raíz:

- Priorización de velocidad de desarrollo sobre calidad desde las fases iniciales
- Falta de experiencia del equipo con herramientas de testing en .NET y React
- Ausencia de una estrategia de testing definida en el proyecto

Solución propuesta — Estrategia por fases:

Fase 1: Unit Tests para handlers CQRS de MediatR (Prioridad: Crítica)

Los handlers de MediatR contienen la lógica de negocio central. Cada handler debe tener al menos un test que valide su comportamiento esperado.

```
 // Ejemplo: CreateRouteCommandHandlerTests.cs
public class CreateRouteCommandHandlerTests
{
    [Fact]
    public async Task Handle_WithValidCommand_ShouldCreateRoute()
    {
        // Arrange
        var mockRepo = new Mock<IRouteRepository>();
        var mockUoW = new Mock<IMobilityUnitOfWork>();
        var handler = new CreateRouteCommandHandler(mockRepo.Object, mockUoW.Object);

        var command = new CreateRouteCommand(
            Name: "Ruta al trabajo",
```

```
            RouteType: "driving",
            OriginLat: 2.9273, OriginLng: -75.2819,
            DestLat: 2.9300, DestLng: -75.2850
        );

        // Act
        var result = await handler.Handle(command, CancellationToken.None);

        // Assert
        Assert.NotEqual(Guid.Empty, result);
        mockRepo.Verify(r => r.AddAsync(It.IsAny<Route>()), Times.Once);
        mockUoW.Verify(u => u.SaveChangesAsync(It.IsAny<CancellationToken>()), Times.Once);
    }
}
```

Alcance mínimo de unit tests (Fase 1):

| Módulo | Handlers a testear | Cobertura mínima | |--------|-------------------|-----------------| | Security | Auth, Users, Roles, Permissions | 80% | | Mobility | Routes, POIs, Favorites, Ratings, Trips | 80% | | Admin | Statistics, Exports, TransportFactors | 70% | | Notifications | Notifications, Templates | 70% |

Fase 2: Integration Tests con Testcontainers para PostgreSQL (Prioridad: Alta)

```
// Ejemplo: RoutesRepositoryTests.cs
public class RoutesRepositoryTests : IAsyncLifetime
{
    private PostgreSqlContainer _postgres;
    private MobilityDbContext _context;

    public async Task InitializeAsync()
    {
        _postgres = new PostgreSqlBuilder()
            .WithImage("postgis/postgis:16-3.5")
            .Build();
        await _postgres.StartAsync();

        // Configurar DbContext con el contenedor
        // Aplicar migraciones Liquibase
    }

    [Fact]
    public async Task AddAsync_ShouldPersistRouteToDatabase()
    {
        // Test real contra PostgreSQL efímero
    }
}
```

Fase 3: API Tests con WebApplicationFactory (Prioridad: Media)

```
// Ejemplo: AuthControllerTests.cs
public class AuthControllerTests : IClassFixture<WebApplicationFactory<Program>>
{
    private readonly HttpClient _client;
```

```
    public AuthControllerTests(WebApplicationFactory<Program> factory)
    {
        _client = factory.CreateClient();
    }

    [Fact]
    public async Task Login_WithValidCredentials_ReturnsToken()
    {
        var response = await _client.PostAsJsonAsync("/api/auth/login",
            new { email = "test@test.com", password = "Password123!" });
        response.EnsureSuccessStatusCode();
    }
}
```

Fase 4: Frontend Tests (Prioridad: Media)

- Unit tests de componentes React con Jest + React Testing Library
- Tests de hooks personalizados
- Tests de servicios API (mock de axios)

Fase 5: E2E Tests (Prioridad: Baja)

- Cypress o Playwright para flujos críticos del frontend
- Detox para flujos críticos de la app móvil

Herramientas seleccionadas:

| Capa | Herramienta | Lenguaje | |------|------------|----------| | Unit tests backend | xUnit + Moq + FluentAssertions | C# | | Integration tests | Testcontainers.PostgreSQL | C# | | API tests | WebApplicationFactory + HttpClient | C# | | Frontend unit tests | Jest + React Testing Library | JS/JSX | | Frontend E2E | Playwright | JS | | Móvil unit tests | Jest | JS/TS |

Métricas de éxito:

| Métrica | Objetivo Fase 1 | Objetivo Fase 2 | |---------|----------------|----------------| | Cobertura de código (backend) | >40% | >65% | | Tests automatizados | >100 | >200 | | Tiempo de ejecución suite | <2 min | <5 min | | Tasa de éxito | 100% | 100% |

Responsable: Backend Lead + QA Engineer **Plazo estimado:** Fase 1: 3 semanas | Fase 2: 2 semanas | Fases 3-5: 4 semanas **Total:** ~9 semanas

---

2.4 Acción Correctiva AC-04: Implementación de pipelines CI/CD automatizados

| Campo | Descripción | |-------|------------| | **ID** | AC-04 | | **Título** | Configuración de integración y entrega continua mediante GitHub Actions | | **Severidad** | 🟠 **Alta** | | **Estado actual** | 🔴 Abierta — Pendiente de ejecución |

Descripción del problema: El proyecto **no cuenta con ninguna pipeline de CI/CD** automatizada. No se encontraron configuraciones de GitHub Actions, GitLab CI, Azure Pipelines ni ninguna otra herramienta de automatización en los cuatro repositorios. Todo el proceso de compilación, prueba y despliegue se realiza manualmente.

Impacto:

- Los errores de compilación se detectan tarde (en lugar de en cada commit)
- Los despliegues son manuales y propensos a errores humanos
- No hay validación automatizada de calidad (linting, análisis estático, seguridad)
- La entrega de valor a producción es lenta e impredecible
- No hay trazabilidad de qué cambio causó qué problema en producción

Causa raíz:

- Priorización de funcionalidad sobre infraestructura de proceso
- Falta de experiencia con herramientas CI/CD en el equipo
- Ausencia de una estrategia de DevOps definida

Solución propuesta:

Pipeline 1: Backend (.NET 9)

```
 # .github/workflows/backend-ci.yml
name: Backend CI

on:
  push:
    branches: [develop, main]
  pull_request:
    branches: [develop, main]

jobs:
  build-and-test:
    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v4

    - name: Setup .NET
      uses: actions/setup-dotnet@v4
      with:
        dotnet-version: '9.0.x'

    - name: Restore dependencies
      run: dotnet restore EcoRuteando.sln

    - name: Build
      run: dotnet build EcoRuteando.sln --no-restore --configuration Release

    - name: Test
      run: dotnet test EcoRuteando.sln --no-build --configuration Release --verbosity norma

    - name: Security audit
      run: dotnet list package --vulnerable --include-transitive

    - name: Upload coverage
      uses: codecov/codecov-action@v4
      with:
```

```
        file: '**/coverage.cobertura.xml'

  docker-build:
    needs: build-and-test
    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v4

    - name: Build Docker image
      run: docker build -t ecoruteando-backend:${{ github.sha }} .

    - name: Smoke test
      run: |
        docker run -d -p 5124:5124 ecoruteando-backend:${{ github.sha }}
        sleep 10
        curl -f http://localhost:5124/api/test/public || exit 1
```

Pipeline 2: Frontend (React/Vite)

```
 # .github/workflows/frontend-ci.yml
name: Frontend CI

on:
  push:
    branches: [develop, main]

jobs:
  build-and-test:
    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v4

    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: '22'
        cache: 'npm'

    - name: Install dependencies
      run: npm ci

    - name: Lint
      run: npm run lint

    - name: Build
      run: npm run build

    - name: Deploy to Vercel (staging)
      if: github.ref == 'refs/heads/develop'
      uses: amondnet/vercel-action@v25
      with:
        vercel-token: ${{ secrets.VERCEL_TOKEN }}
        vercel-org-id: ${{ secrets.VERCEL_ORG_ID }}
```

```
        vercel-project-id: ${{ secrets.VERCEL_PROJECT_ID }}
        vercel-args: '--pre'
```

Pipeline 3: Base de Datos (Liquibase)

```
 # .github/workflows/db-ci.yml
name: Database CI

on:
  pull_request:
    paths:
      - '01_ddl/**'
      - '02_dml/**'
      - 'changelog/**'

jobs:
  validate-changesets:
    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v4

    - name: Validate Liquibase changelog
      uses: liquibase-github-actions@v4
      with:
        operation: validate
        arguments: --changeLogFile=changelog/db.changelog-master.yaml

    - name: Test against PostgreSQL
      run: |
        docker run -d --name postgres-test -e POSTGRES_PASSWORD=postgres -p 5432:5432 postg
        sleep 10
        # Aplicar changelog y verificar
```

Pipeline 4: Móvil (Expo/React Native)

```
 # .github/workflows/mobile-ci.yml
name: Mobile CI

on:
  push:
    branches: [develop, main]

jobs:
  build-preview:
    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v4

    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: '22'
```

```
    - name: Install EAS CLI
      run: npm install -g eas-cli

    - name: Build with EAS
      run: eas build --profile preview --platform android --non-interactive
      env:
        EXPO_TOKEN: ${{ secrets.EXPO_TOKEN }}
```

Estrategia de ramas y ambientes:

```
main (producción)
 └── develop (integración)
      └── qa (calidad)
           └── feature/* (desarrollo)
```

| Rama | Acción CI | Despliegue | |------|-----------|-----------| | `feature/*` | Build + Test | Ninguno | | `qa` | Build + Test + Deploy | Ambiente QA | | `develop` | Build + Test + Deploy | Ambiente Staging | | `main` | Build + Test + Deploy | Producción |

Responsable: DevOps Engineer **Plazo estimado:** 2-3 semanas (incluyendo configuración de secrets en GitHub)

---

3. Matriz de Acciones Preventivas

Las acciones preventivas mitigan **riesgos futuros** que podrían impactar la seguridad, estabilidad o disponibilidad de la plataforma.

3.1 Acción Preventiva AP-01: Prevención de filtración de secretos

| Campo | Descripción | |-------|-------------| | **ID** | AP-01 | | **Riesgo** | Filtración o exposición accidental de secretos en repositorios | | **Probabilidad** | 🟠 Media-Alta (ya ocurrió una vez — AC-01) | | **Impacto** | 🔴 Crítico (facturación, suplantación, phishing) | | **Riesgo inherente** | 🔴 Muy Alto |

Medidas preventivas:

| # | Medida | Herramienta | Estado | |---|--------|-------------|--------| | 1 | **Secret Scanning en GitHub** — Detección automática de secretos en commits y PRs | GitHub Secret Scanning | ⬜ Pendiente | | 2 | **Push Protection** — Bloquear pushes que contengan secretos conocidos | GitHub Push Protection | ⬜ Pendiente | | 3 | **Pre-commit hooks locales** — Bloquear commits con patrones de secretos antes de que salgan del developer machine | `pre-commit` + `gitleaks` | ⬜ Pendiente | | 4 | **Auditoría periódica con Gitleaks** — Escaneo programado del historial completo | `gitleaks detect --source .` (cron semanal) | ⬜ Pendiente | | 5 | **Vault / Secret Manager** — Almacenar secretos de producción en un servicio dedicado | HashiCorp Vault / AWS Secrets Manager / Azure Key Vault | ⬜ Pendiente | | 6 | **Política de rotación automática** — Rotar API keys cada 90 días | Google Cloud IAM + script | ⬜ Pendiente | | 7 | **Capacitación del equipo** — Workshop sobre gestión de secretos | Documentación interna | ⬜ Pendiente |

Mecanismo de monitoreo:

| Indicador | Frecuencia | Umbral de alerta | |----------|----------|----------| | Secretos detectados en PRs | Cada PR | 0 tolerancia | | Secretos en historial (gitleaks scan) | Semanal | 0 tolerancia | | API keys sin rotar >90 días | Mensual | 0 tolerancia |

Responsable: DevOps Engineer + Security Lead **Plazo:** 1 semana para medidas 1-3; 1 mes para medida 5

3.2 Acción Preventiva AP-02: Prevención de pérdida o corrupción de datos

| Campo | Descripción | |-------|----------| | **ID** | AP-02 | | **Riesgo** | Pérdida o corrupción de datos en PostgreSQL ante fallos del servidor | | **Probabilidad** | 🟡 Media (fallos de hardware, errores humanos, desastres naturales) | | **Impacto** | 🔴 Crítico (pérdida de datos de usuarios, rutas, historiales) | | **Riesgo inherente** | 🟠 Alto |

Medidas preventivas:

| # | Medida | Detalle | Frecuencia | |---|--------|---------|----------| | 1 | **Backups automáticos en Render** | Habilitar backups automatizados del servicio PostgreSQL gestionado | Diario | | 2 | **Backups incrementales con pg_dump** | Script de backup automático a almacenamiento externo (S3/GCS) | Cada 6 horas | | 3 | **Backups completos semanales** | `pg_dump` completo comprimido con retención de 30 días | Semanal | | 4 | **Verificación de integridad de backups** | Restaurar backup en entorno aislado y validar datos | Semanal | | 5 | **WAL archiving** | Habilitar Write-Ahead Logging para point-in-time recovery | Continuo | | 6 | **Réplica de lectura** | Configurar réplica de PostgreSQL para disaster recovery | Continuo | | 7 | **Runbook de recuperación** | Documentar procedimiento de restauración paso a paso | Revisión trimestral | | 8 | **Migraciones versionadas** | Liquibase garantiza que los cambios de esquema sean reversibles | Cada cambio |

Script de backup automatizado:

```
 #!/bin/bash
# scripts/backup-database.sh
BACKUP_DIR="/backups/ecoruteando"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
FILENAME="ecoruteando_${TIMESTAMP}.sql.gz"

# Backup completo
docker compose exec postgres pg_dump -U postgres ecoRuteando_db | gzip > ${BACKUP_DIR}/${FI

# Verificar tamaño mínimo (debe ser > 1MB)
FILESIZE=$(stat -f%z "${BACKUP_DIR}/${FILENAME}" 2>/dev/null || stat -c%s "${BACKUP_DIR}/${
if [ "$FILESIZE" -lt 1048576 ]; then
    echo "ERROR: Backup sospechosamente pequeño ($FILESIZE bytes)"
    exit 1
fi

# Retener solo últimos 30 backups
ls -t ${BACKUP_DIR}/ecoruteando_*.sql.gz | tail -n +31 | xargs rm -f

echo "Backup completado: ${FILENAME} (${FILESIZE} bytes)"
```

Mecanismo de monitoreo:

| Indicador | Frecuencia | Umbral de alerta | |----------|----------|----------------| | Éxito de backup automático | Diario | 100% éxito | | Tamaño del backup | Diario | > 1 MB | | Tiempo de restauración de prueba | Semanal | < 30 minutos | | Espacio disponible en almacenamiento | Diario | > 20% libre |

Responsable: DevOps Engineer + DBA (part-time) **Plazo:** 1 semana para medidas 1-3; 2 semanas para medidas 4-8

3.3 Acción Preventiva AP-03: Prevención de caídas del backend e indisponibilidad de APIs externas

| Campo | Descripción | |-------|------------| | **ID** | AP-03 | | **Riesgo** | Caídas del backend o indisponibilidad de APIs externas (Google Maps / Google Weather) | | **Probabilidad** | 🟡 Media (Google Maps tiene SLA del 99.9%, pero interrupciones ocurren) | | **Impacto** | 🟠 Alto (funcionalidad de planificación de rutas inoperativa) | | **Riesgo inherente** | 🟠 Alto |

Medidas preventivas:

| # | Medida | Detalle | |---|--------|---------| | 1 | **Health checks activos** | Endpoints `/health` en backend y `/api/test/public` para monitoreo | | 2 | **Circuit Breaker pattern** | Polly para cortar llamadas a APIs externas tras N fallos consecutivos | | 3 | **Retry con backoff exponencial** | Reintentar llamadas a Google Maps con espera progresiva | | 4 | **Caché de respuestas** | Tabla `integrations.api_cache` con TTL para respuestas de Google Maps | | 5 | **Timeout estricto** | Configurar timeouts de 5-10s en HttpClient para llamadas externas | | 6 | **Graceful degradation** | Si Google Maps falla, ofrecer cálculo local de distancia (Haversine) | | 7 | **Health monitoring** | Uptime Robot / BetterStack para monitoreo 24/7 del backend | | 8 | **Auto-restart** | `restart: unless-stopped` en docker-compose (ya implementado) | | 9 | **Load balancing** | Considerar múltiples instancias del backend en Render | | 10 | **Alertas de API key quota** | Notificar cuando Google Cloud marque cuota al 80% |

Implementación del Circuit Breaker:

```
 // Ejemplo: Polly Circuit Breaker para Google Maps
services.AddHttpClient<IGoogleMapsService, GoogleMapsService>(client =>
{
    client.BaseAddress = new Uri("https://maps.googleapis.com/");
    client.Timeout = TimeSpan.FromSeconds(10);
})
.AddPolicyHandler(Policy<HttpResponseMessage>
    .Handle<HttpRequestException>()
    .OrResult(r => !r.IsSuccessStatusCode)
    .CircuitBreakerAsync(
        handledEventsAllowedBeforeBreaking: 3,
        durationOfBreak: TimeSpan.FromSeconds(30))
)
.AddPolicyHandler(GetRetryPolicy()); // Retry con backoff exponencial

// Retry policy
static IAsyncPolicy<HttpResponseMessage> GetRetryPolicy()
{
```

```
    return HttpPolicyExtensions
        .HandleTransientHttpError()
        .WaitAndRetryAsync(
            retryCount: 3,
            sleepDurationProvider: retryAttempt => TimeSpan.FromSeconds(Math.Pow(2, retryAt
            onRetry: (outcome, timespan, retryCount, context) =>
            {
                logger.LogWarning("Reintentando Google Maps (intento {RetryCount}) tras {De
                    retryCount, timespan.TotalSeconds);
            });
}
```

Mecanismo de monitoreo:

| Indicador | Herramienta | Umbral de alerta | |----------|----------|----------| | Uptime del backend | Uptime Robot / BetterStack | < 99.5% mensual | | Tiempo de respuesta API | Application Insights / Seq | > 2s promedio | | Errores 5xx | Logs + monitoreo | > 1% de requests | | Circuit breaker state | Logs estructurados | BREAKING state | | Cuota Google Maps | Google Cloud Console | > 80% consumido |

Responsable: Backend Lead + DevOps **Plazo:** 2 semanas

3.4 Acción Preventiva AP-04: Prevención de regresiones y fallos de esquema

| Campo | Descripción | |-------|----------| | **ID** | AP-04 | | **Riesgo** | Regresiones en la lógica de negocio del backend o fallos de esquema al aplicar cambiosets de Liquibase | | **Probabilidad** | 🟡 Media (sin pruebas automatizadas — ver AC-03) | | **Impacto** | 🟠 Alto (datos inconsistentes, downtime, rollback forzoso) | | **Riesgo inherente** | 🟠 Alto |

Medidas preventivas:

| # | Medida | Detalle | |---|--------|---------| | 1 | **Validación de changelogs antes de merge** | Pipeline CI ejecuta `liquibase validate` en cada PR | | 2 | **Tests de migración** | Aplicar changelog completo en BD limpia dentro del CI | | 3 | **Changeset de rollback** | Incluir `rollback` blocks en cada changeset Liquibase | | 4 | **Code review obligatorio** | Ningún PR a `develop` / `main` sin aprobación de al menos 1 reviewer | | 5 | **Feature flags** | Funcionalidades nuevas detrás de feature flags para activación gradual | | 6 | **Staging environment** | Entorno de staging con datos de producción anonimizados | | 7 | **Canary deployments** | Desplegar cambios a un subconjunto de usuarios primero | | 8 | **Runbook de rollback** | Procedimiento documentado para revertir despliegues en < 5 minutos | | 9 | **Análisis estático** | SonarQube / Roslyn analyzers para detectar code smells y bugs potenciales | | 10 | **Dependency scanning** | Dependabot para detectar vulnerabilidades en dependencias |

Ejemplo de changeset con rollback:

```
<changeSet id="020-add-user-status-index" author="dev">
    <preConditions onFail="MARK_RAN">
        <not><indexExists indexName="idx_users_status" tableName="users" schemaName="securi
    </preConditions>
    <createIndex indexName="idx_users_status" tableName="users" schemaName="security">
        <column name="status"/>
```

```
    </createIndex>
    <rollback>
        <dropIndex indexName="idx_users_status" tableName="users" schemaName="security"/>
    </rollback>
</changeSet>
```

Mecanismo de monitoreo:

| Indicador | Frecuencia | Umbral de alerta | |----------|----------|----------------| | Fallos de Liquibase en CI | Cada PR | 0 tolerancia | | Regresiones detectadas en QA | Semanal | < 2 por sprint | | Rollbacks en producción | Mensual | < 1 por mes | | Bugs reportados post-release | Por release | < 5 por release |

Responsable: Backend Lead + QA Engineer **Plazo:** 2 semanas (medidas 1-4); 1 mes (medidas 5-10)

---

4. Plan de Mejoramiento y Optimización Técnica

Las acciones de mejoramiento optimizan la arquitectura, observabilidad y capacidades funcionales de la plataforma a mediano plazo.

4.1 Mejora M-01: Configuración de logs centralizados con Serilog

| Campo | Descripción | |-------|-------------| | **ID** | M-01 | | **Área** | Observabilidad | | **Complejidad** | 🟢 Baja | | **Impacto** | 🟠 Alto |

Situación actual: Serilog está instalado como dependencia (`Serilog.AspNetCore` 8.0.3, `Serilog.Sinks.Console` 5.0.1) pero **no está configurado en** `Program.cs`. Los logs actuales usan el provider por defecto de ASP.NET Core (`ILogger<T>` → consola), sin enriquecimiento estructurado, sin sinks persistentes y sin correlación de requests.

Beneficios esperados:

- Logs estructurados (JSON) fáciles de consultar y analizar
- Correlación de requests mediante `RequestID` para tracing distribuido
- Persistencia de logs en archivos/Splunk/Seq para auditoría forense
- Filtración por nivel de severidad en tiempo real
- Enrichment automático con contexto (usuario, módulo, exception)

Solución propuesta:

Instalación de sinks adicionales:

```
 dotnet add package Serilog.Sinks.File
dotnet add package Serilog.Sinks.Seq
dotnet add package Serilog.Enrichers.Environment
dotnet add package Serilog.Enrichers.Thread
dotnet add package Serilog.AspNetCore
```

Configuración en `Program.cs`:

```
// Configuración de Serilog
Log.Logger = new LoggerConfiguration()
    .ReadFrom.Configuration(configuration)
    .Enrich.FromLogContext()
    .Enrich.WithEnvironmentName()
    .Enrich.WithMachineName()
    .Enrich.WithThreadId()
    .WriteTo.Console(outputTemplate:
        "[{Timestamp:HH:mm:ss} {Level:u3}] {SourceContext} {Message:lj}{Exception}{NewLine}
    .WriteTo.File(
        path: "logs/ecoruteando-.log",
        rollingInterval: RollingInterval.Day,
        retainedFileCountLimit: 30,
        outputTemplate: "{Timestamp:yyyy-MM-dd HH:mm:ss.fff zzz} [{Level:u3}] {SourceContex
    .WriteTo.Seq(serverUrl: "http://localhost:5341") // Seq para logs estructurados
    .CreateLogger();

try
{
    Log.Information("Iniciando EcoRuteando API");
    // ... resto de configuración
    app.Run();
}
catch (Exception ex)
{
    Log.Fatal(ex, "La aplicación terminó inesperadamente");
}
finally
{
    Log.CloseAndFlush();
}
```

Configuración en `appsettings.json`:

```
{
  "Serilog": {
    "MinimumLevel": {
      "Default": "Information",
      "Override": {
        "Microsoft": "Warning",
        "Microsoft.Hosting.Lifetime": "Information",
        "System": "Warning"
      }
    },
    "WriteTo": [
      { "Name": "Console" },
      { "Name": "File", "Args": { "path": "logs/ecoruteando-.log", "rollingInterval": "Day"
    ],
    "Enrich": ["FromLogContext", "WithEnvironmentName", "WithMachineName"],
    "Properties": { "Application": "EcoRuteando.API" }
  }
}
```

Indicadores de éxito:

| Indicador | Objetivo | |----------|---------| | Logs estructurados en formato JSON | 100% de logs | | Correlación de requests con RequestID | 100% de requests HTTP | | Retención de logs | 30 días mínimo | | Búsqueda de errores en logs | < 1 minuto con Seq/ELK |

Responsable: Backend Lead **Plazo:** 3-5 días

4.2 Mejora M-02: Implementación de Rate Limiting en endpoints de autenticación

| Campo | Descripción | |-------|-------------| | **ID** | M-02 | | **Área** | Seguridad | | **Complejidad** | 🟢 Baja-Media | | **Impacto** | 🟠 Alto |

Situación actual: Los endpoints de autenticación (`POST /api/auth/login` , `POST /api/auth/refresh`) **no tienen limitación de tasa** implementada. Un atacante puede realizar intentos ilimitados de fuerza bruta contra credenciales de usuarios.

Beneficios esperados:

- Mitigación de ataques de fuerza bruta contra contraseñas
- Protección contra ataques de denegación de servicio (DoS) a nivel de aplicación
- Cumplimiento parcial de OWASP ASVS V2.2 (Rate Limiting)
- Registro de intentos fallidos para detección de intrusos

Solución propuesta:

Instalación:

```
dotnet add package System.Threading.RateLimiting
```

Configuración en `Program.cs` :

```
using System.Threading.RateLimiting;

// Configurar Rate Limiting
builder.Services.AddRateLimiter(options =>
{
    options.RejectionStatusCode = StatusCodes.Status429TooManyRequests;

    // Policy para endpoints de login: 5 intentos por minuto por IP
    options.AddPolicy("login", httpContext =>
        RateLimitPartition.GetFixedWindowLimiter(
            partitionKey: httpContext.Connection.RemoteIpAddress?.ToString() ?? "unknown",
            factory: partition => new FixedWindowRateLimiterOptions
            {
                PermitLimit = 5,
                Window = TimeSpan.FromMinutes(1),
                QueueLimit = 0
            }));

    // Policy para refresh token: 10 por minuto por IP
    options.AddPolicy("refresh", httpContext =>
```

```
        RateLimitPartition.GetFixedWindowLimiter(
            partitionKey: httpContext.Connection.RemoteIpAddress?.ToString() ?? "unknown",
            factory: partition => new FixedWindowRateLimiterOptions
            {
                PermitLimit = 10,
                Window = TimeSpan.FromMinutes(1),
                QueueLimit = 0
            }));

    // Policy global: 100 por minuto por usuario
    options.GlobalLimiter = PartitionedRateLimiter.Create<HttpContext, string>(httpContext
        RateLimitPartition.GetFixedWindowLimiter(
            partitionKey: httpContext.User.Identity?.Name ?? httpContext.Connection.RemoteI
            factory: partition => new FixedWindowRateLimiterOptions
            {
                PermitLimit = 100,
                Window = TimeSpan.FromMinutes(1)
            }));
});

// Aplicar rate limiter
app.UseRateLimiter();
```

Aplicación en controllers:

```
 [ApiController]
[Route("api/auth")]
public class AuthController : ControllerBase
{
    [HttpPost("login")]
    [EnableRateLimiting("login")]
    public async Task<IActionResult> Login([FromBody] LoginRequest request)
    {
        // ...
    }

    [HttpPost("refresh")]
    [EnableRateLimiting("refresh")]
    public async Task<IActionResult> Refresh([FromBody] RefreshRequest request)
    {
        // ...
    }
}
```

Indicadores de éxito:

| Indicador | Objetivo | |----------|---------| | Requests bloqueados por rate limiter | Registrados en logs | | Intentos de fuerza bruta mitigados | 100% bloqueados tras 5 intentos/min | | Impacto en usuarios legítimos | < 0.1% de falsos positivos | | Tiempo de respuesta adicional | < 1ms overhead |

Responsable: Backend Lead **Plazo:** 2-3 días

4.3 Mejora M-03: Estrategia de pruebas automatizadas prioritaria

| Campo | Descripción | |-------|-------------| | **ID** | M-03 | | **Área** | Calidad de Software | | **Complejidad** | 🟠 Media-Alta | | **Impacto** | 🔴 Crítico |

> **Nota:** Esta mejora está detallada como acción correctiva AC-03. Se replica aquí como parte del plan de mejora continua para asegurar su seguimiento integrado.

Enfoque prioritario:

1. Unit tests de handlers MediatR (mayor ROI)
2. Integration tests con Testcontainers para PostgreSQL
3. API tests con WebApplicationFactory
4. Frontend tests con Jest + React Testing Library

Indicadores de éxito:

| Métrica | Baseline | Objetivo 3 meses | Objetivo 6 meses | |---------|----------|------------------|------------------| | Cobertura backend | 0% | > 40% | > 65% | | Cobertura frontend | 0% | > 20% | > 40% | | Tests automatizados | 0 | > 150 | > 300 | | Tiempo de ejecución | N/A | < 3 min | < 5 min | | Tasa de éxito | N/A | 100% | 100% |

Responsable: Backend Lead + QA Engineer **Plazo:** 9 semanas (alineado con AC-03)

4.4 Mejora M-04: Habilitación de push notifications reales (FCM/APNs)

| Campo | Descripción | |-------|-------------| | **ID** | M-04 | | **Área** | Funcionalidad Móvil | | **Complejidad** | 🟠 Media | | **Impacto** | 🟡 Medio |

Situación actual: El sistema de notificaciones tiene la infraestructura completa:

- Tabla `notifications.device_tokens` para almacenar tokens de dispositivos
- Endpoint `POST /api/notifications/device-token` para registrar tokens
- Modelo `NotificationType` con tipos `info`, `warning`, `success`, `error`, `weather`, `social`
- Módulo Notifications con CRUD completo

Sin embargo, **el envío real de push notifications no está implementado**. Los tokens se registran pero nunca se usan para enviar notificaciones push a los dispositivos.

Beneficios esperados:

- Notificaciones en tiempo real de alertas climáticas a lo largo de rutas
- Notificaciones de interacciones sociales (calificaciones, reportes)
- Notificaciones de sistema (nuevas funcionalidades, mantenimiento)
- Mejora significativa de la retención de usuarios (engagement)

Solución propuesta:

Fase 1: Integración con FCM (Firebase Cloud Messaging)

```
 // Infrastructure/Services/PushNotificationService.cs
public class PushNotificationService : IPushNotificationService
{
```

```
    private readonly IFirebaseMessaging _firebaseMessaging;
    private readonly ILogger<PushNotificationService> _logger;

    public async Task SendPushAsync(string deviceToken, PushNotificationRequest request)
    {
        var message = new Message()
        {
            Token = deviceToken,
            Notification = new Notification()
            {
                Title = request.Title,
                Body = request.Body
            },
            Data = request.Data,
            Android = new AndroidConfig()
            {
                Priority = Priority.High
            },
            Apns = new ApnsConfig()
            {
                Aps = new Aps()
                {
                    Sound = "default",
                    Badge = 1
                }
            }
        };

        try
        {
            await _firebaseMessaging.SendAsync(message);
        }
        catch (FirebaseException ex)
        {
            _logger.LogError(ex, "Error enviando push a {Token}", deviceToken);
            // Remover token inválido
        }
    }
}
```

Fase 2: Configuración de Firebase

1. Crear proyecto en Firebase Console
2. Descargar `google-services.json` (Android) y `GoogleService-Info.plist` (iOS)
3. Configurar FCM en el backend con service account key
4. Integrar `expo-notifications` en la app móvil

Fase 3: Integración en la app móvil

```
 // app/_layout.tsx
import * as Notifications from 'expo-notifications';

Notifications.setNotificationHandler({
  handleNotification: async () => ({
    shouldShowAlert: true,
```

```
    shouldPlaySound: true,
    shouldSetBadge: true,
  }),
});

async function registerForPushNotifications() {
  const { status } = await Notifications.requestPermissionsAsync();
  if (status !== 'granted') return null;

  const token = (await Notifications.getExpoPushTokenAsync()).data;

  // Registrar token en backend
  await apiClient.post('/notifications/device-token', {
    token,
    platform: Platform.OS
  });

  return token;
}
```

Indicadores de éxito:

| Indicador | Objetivo | |----------|---------| | Tokens registrados | > 50% de usuarios activos | | Tasa de entrega push | > 95% | | Clic en notificación | > 15% | | Notificaciones push enviadas/día | > 100 |

Responsable: Backend Lead + Mobile Developer **Plazo:** 2-3 semanas

---

5. Conclusiones y seguimiento

5.1 Resumen ejecutivo

La auditoría técnica del proyecto EcoRuteando revela una plataforma con una **arquitectura de software sólida** (DDD + Clean Architecture con CQRS/MediatR), un **87% de funcionalidades completamente implementadas** y una **buena separación de responsabilidades** entre sus cinco bounded contexts. Sin embargo, se identificaron **brechas críticas en seguridad, calidad de software e infraestructura DevOps** que requieren atención inmediata antes de escalar la plataforma a producción.

5.2 Cuadro resumen de acciones

| ID | Tipo | Título | Severidad | Prioridad | Plazo | Responsable | |----|------|--------|-----------|-----------|-------|-------------| | AC-01 | Correctiva | Rotación de credenciales filtradas | 🔴 Crítica | Inmediata | 1-2 sem | DevOps | | AC-02 | Correctiva | Restablecer servicio SMTP | 🟠 Alta | Alta | 1 sem | Backend Lead | | AC-03 | Correctiva | Implementar suite de pruebas | 🟠 Alta | Alta | 9 sem | Backend + QA | | AC-04 | Correctiva | Implementar CI/CD pipelines | 🟠 Alta | Alta | 2-3 sem | DevOps | | AP-01 | Preventiva | Prevención filtración de secretos | 🔴 Crítica | Alta | 1 sem | DevOps | | AP-02 | Preventiva | Prevención pérdida de datos | 🔴 Crítica | Media | 1-2 sem | DevOps | | AP-03 | Preventiva | Prevención caídas backend/APIs | 🟠 Alta | Media | 2 sem | Backend Lead | | AP-04 | Preventiva | Prevención regresiones/errores esquema | 🟠 Alta | Media | 2-4 sem | Backend + QA | | M-01 | Mejora | Logs centralizados Serilog | 🟠 Alta | Media | 3-5 días | Backend Lead | | M-02 |

Mejora | Rate Limiting auth endpoints | 🟠 Alta | Media | 2-3 días | Backend Lead | | M-03 | Mejora | Estrategia pruebas automatizadas | 🔴 Crítica | Alta | 9 sem | Backend + QA | | M-04 | Mejora | Push notifications FCM/APNs | 🟡 Media | Baja | 2-3 sem | Backend + Mobile |

5.3 Secuencia de ejecución recomendada

Semana 1-2 (Inmediato):

1. ✅ AC-01 — Rotar credenciales y limpiar Git
2. ✅ AP-01 — Configurar secret scanning
3. ✅ AC-02 — Corregir SMTP
4. ✅ AP-02 — Configurar backups automáticos

Semana 3-4 (Corto plazo): 5. ✅ AC-04 — Implementar CI/CD pipelines 6. ✅ M-01 — Configurar Serilog 7. ✅ M-02 — Implementar Rate Limiting

Semana 5-12 (Medio plazo): 8. ✅ AC-03 + M-03 — Implementar suite de pruebas 9. ✅ AP-03 — Circuit Breaker y monitoreo 10. ✅ AP-04 — Validación de cambiosets y code review

Semana 13+ (Continuo): 11. ✅ M-04 — Push notifications 12. ✅ Mejoras iterativas basadas en métricas

5.4 Métricas de seguimiento del plan

| Categoría | Métrica | Baseline | Objetivo 3 meses | Objetivo 6 meses | |----------|--------|---------|-----------|-----------|-----------------| | Seguridad | Secretos en repositorio | 4+ | 0 | 0 | | Seguridad | Endpoints con rate limiting | 0% | 100% (auth) | 100% (global) | | Calidad | Cobertura de pruebas backend | 0% | > 40% | > 65% | | Calidad | Tiempo de detección de bugs | Manual | < 1 día | < 4 horas | | DevOps | Frecuencia de despliegues | Manual | 1/semana | Diario | | DevOps | Tiempo de recuperación (MTTR) | N/A | < 1 hora | < 30 min | | Observabilidad | Logs estructurados | 0% | 100% | 100% | | Disponibilidad | Uptime backend | Monitoreado | > 99.5% | > 99.9% | | Funcionalidad | Push notifications | No implementado | Beta | 100% |

5.5 Riesgos del plan de mejora

| Riesgo | Probabilidad | Impacto | Mitigación | |--------|-------------|---------|------------| | Resistencia del equipo a nuevas herramientas | Media | Medio | Capacitación, documentación, pair programming | | Sobrecarga de trabajo (feature freeze) | Alta | Alto | Fases progresivas, priorización estricta | | Presupuesto insuficiente para herramientas | Baja | Medio | Priorizar herramientas open-source | | Cambios en requisitos del cliente | Media | Alto | Planificación flexible, revisión quincenal | | Dependencia de servicios externos (Render, Vercel) | Baja | Alto | Evaluación de alternativas, multi-cloud strategy |

5.6 Gobernanza y revisión

| Actividad | Frecuencia | Participantes | Output | |----------|----------|---------------|--------| | Daily standup | Diaria | Equipo técnico | Estado de tareas | | Revisión de acciones correctivas | Semanal | Tech Lead + DevOps | Dashboard de progreso | | Retrospectiva de mejora continua | Quincenal | Equipo completo | Acciones de mejora | | Auditoría de seguridad | Mensual | Security Lead | Reporte de

hallazgos | | Revisión de métricas | Mensual | Tech Lead + Management | Dashboard ejecutivo | | Revisión del plan completo | Trimestral | Stakeholders | Plan actualizado |

5.7 Aprobación del documento

| Rol | Nombre | Firma | Fecha | |-----|--------|-------|-------| | Project Manager | _____ | _____ | //2026 | | Tech Lead / Arquitecto | _____ | _____ | //2026 | | DevOps Engineer | _____ | _____ | //2026 | | QA Engineer | _____ | _____ | //2026 | | Security Lead | _____ | _____ | //2026 |

Documento generado como parte del plan de Mejora Continua de EcoRuteando — v1.0 — 13 de octubre de 2026